CT-159 · DATA STRUCTURES

Advanced Sorting

Lecture 7 — Merge Sort & Quick Sort (Divide & Conquer)

Presenter: Dr. Muhammad Kamran

SECTION 01

Divide & Conquer

Break a big problem into smaller ones, solve those, then combine.

The Divide & Conquer Idea

- Divide: split the problem into smaller sub-problems of the same kind.
- Conquer: solve each sub-problem recursively (tiny ones are trivial).
- Combine: merge the sub-answers into the answer for the whole.
- Merge Sort and Quick Sort both use this to beat the $O(n^2)$ simple sorts.

Both run in O(n log n) on average — a big jump over Selection / Bubble / Insertion.

SECTION 02

Merge Sort

Split down to single elements, then merge back in order.

What is Merge Sort?

Divide the array in half until each piece has one element, then merge the pieces back in sorted order.

- A stable, divide-and-conquer sort.
- A single element is already sorted — that's the base case.
- The real work is in merging two sorted halves into one.

Merge Sort — Algorithm

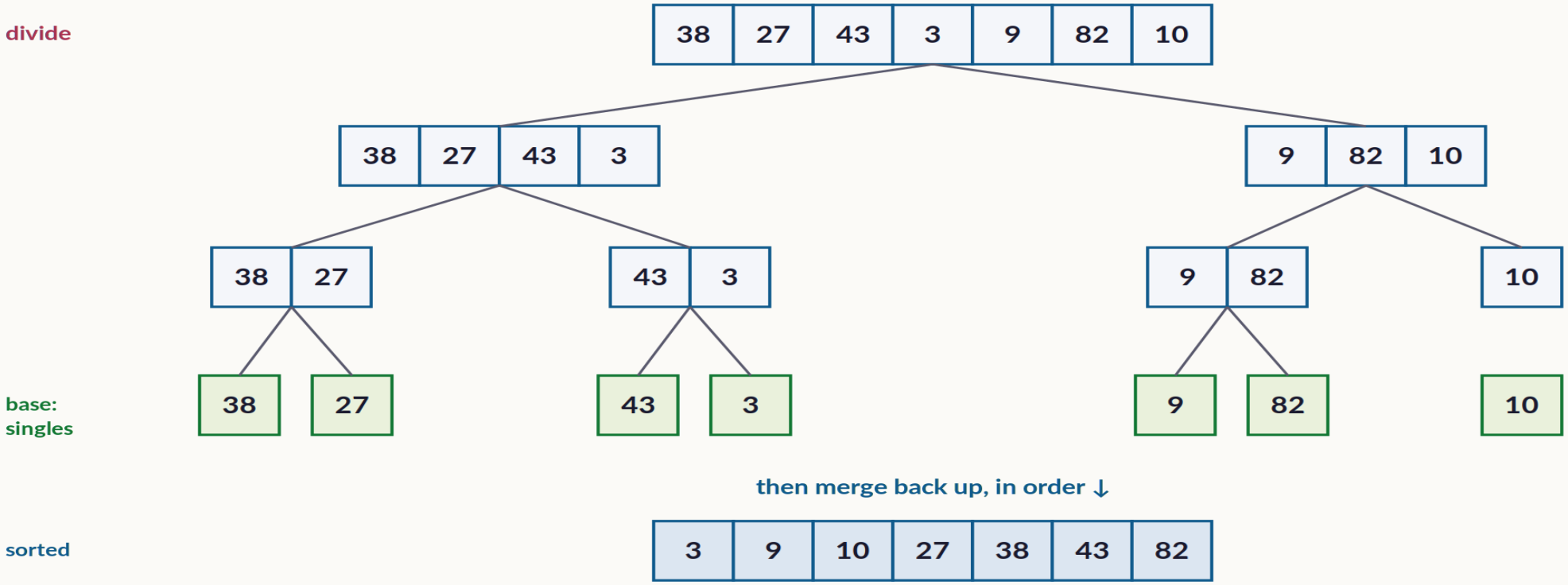
Steps

1. If the array has ≤ 1 element, it's sorted
2. Find the middle, split into left & right
3. Recursively merge-sort the left half
4. Recursively merge-sort the right half
5. Merge the two sorted halves

```
// pseudocode
mergeSort(arr, l, r):
    if l >= r: return           //
base case
    mid = (l + r) / 2
    mergeSort(arr, l, mid)
    mergeSort(arr, mid+1, r)
    merge(arr, l, mid, r)
```

Merge Sort — Worked Example

Splitting [38,27,43,3,9,82,10] down to singles, then merging back sorted.



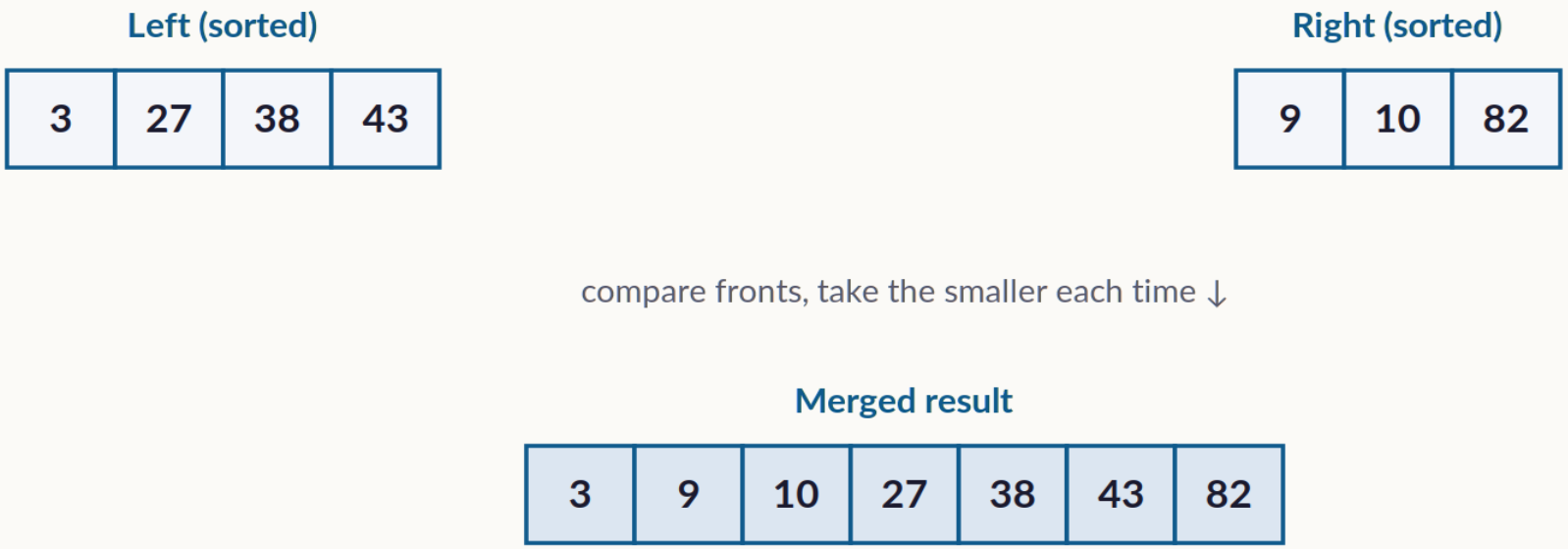
Divide the array in half repeatedly until each piece has one element (already sorted).

Then merge pairs back together in sorted order, all the way up to the full array.

log n levels of splitting × O(n) work to merge each level → O(n log n).

The Merge Step

Two sorted halves combine in O(n) by repeatedly taking the smaller front element.



Two already-sorted lists merge in O(n): walk both fronts, always copying the smaller value.

When one side empties, copy the rest of the other side.

The Merge — Algorithm

Steps

1. Two pointers at the fronts of left & right
2. Compare them; copy the smaller to output
3. Advance the pointer you copied from
4. Repeat until one side is empty
5. Copy any remaining elements over

```
// pseudocode
merge(L, R):
    i = j = 0; out = []
    while i < len(L) and j < len(R):
        if L[i] <= R[j]:
            out.append(L[i]); i++
        else:
            out.append(R[j]); j++
    append leftover of L (from i)
    append leftover of R (from j)
    return out
```

Merge Sort — Complexity

Best	Average	Worst	Space
O(n log n)	**O(n log n)**	**O(n log n)**	**O(n)**

Consistently O(n log n) — log n levels of splitting, O(n) to merge each level. The trade-off is O(n) extra space for the merge, and it's stable.

SECTION 03

Quick Sort

Pick a pivot, partition around it, then recurse on each side.

What is Quick Sort?

Choose a pivot, partition the array so smaller values go left and larger go right, then recurse.

- A divide-and-conquer sort that works in place (no extra array).
- After each partition, the pivot sits in its final sorted position.
- Very fast in practice — often the default library sort.

Choosing a Pivot

First / last element	Simplest, but $O(n^2)$ on already-sorted input.
Random element	Avoids the sorted-input trap on average.
Median-of-three	Take the median of first, middle, last — a good practical choice.

The pivot choice is what decides whether quick sort stays fast or degrades to $O(n^2)$.

Quick Sort — Algorithm

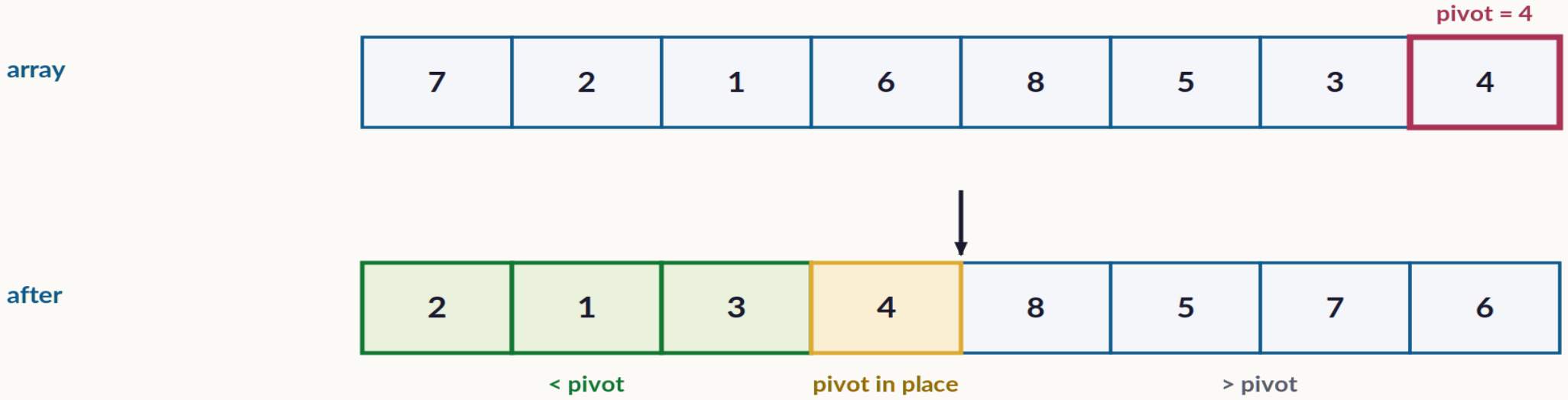
Steps

1. If low < high:
2. Partition around a pivot → pivot index p
3. The pivot is now in its final place
4. Recurse quick-sort on the left part
5. Recurse quick-sort on the right part

```
// pseudocode
quickSort(arr, low, high):
    if low < high:
        p = partition(arr, low,
high)
        quickSort(arr, low, p-1)
        quickSort(arr, p+1, high)
```

The Partition Step

Everything smaller than the pivot moves left, everything larger moves right.



Partition puts the pivot in its final sorted position:

everything smaller ends up on its left, everything larger on its right.

Then quicksort recurses on the left part and the right part separately.

Good pivots → $O(n \log n)$ average. A bad pivot (already-sorted worst case) → $O(n^2)$.

Partition — Algorithm (Lomuto)

Steps

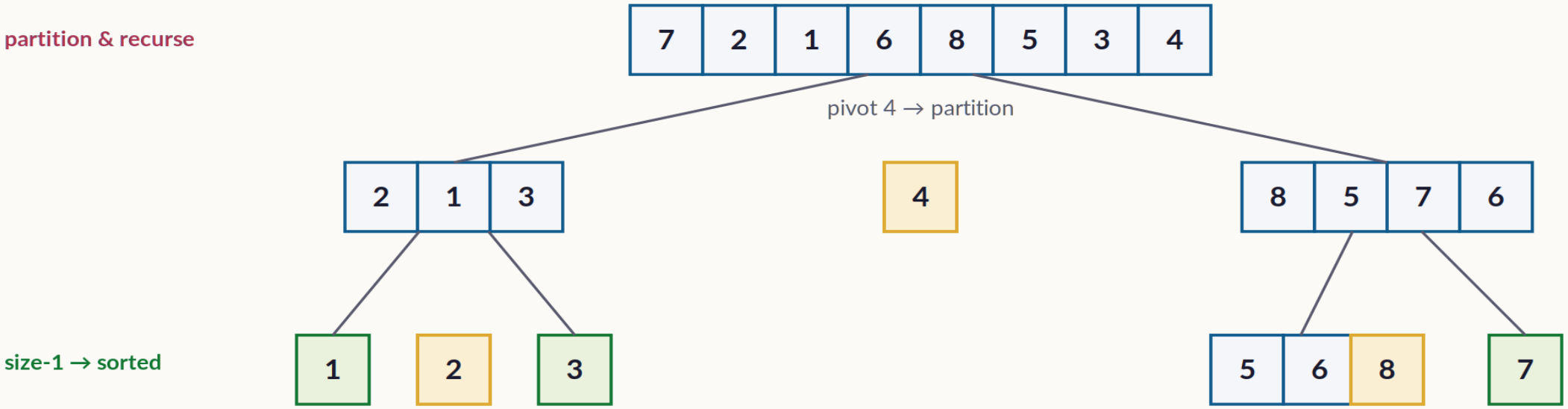
1. Choose pivot = last element
2. i marks the boundary of smaller values
3. Scan j; if arr[j] < pivot, swap into i, i++
4. After the scan, swap pivot into position i
5. Return i as the pivot's final index

```
// pseudocode

partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j = low to high-1:
        if arr[j] < pivot:
            i++
            swap(arr[i], arr[j])
    swap(arr[i+1], arr[high])
    return i + 1
```

Quick Sort — Worked Example

Each partition fixes one pivot; quick sort then recurses on each side.



Each partition fixes one pivot (amber), then quicksort recurses on each side.

Unlike merge sort, the sorting happens during the split — there is no separate merge step.

Balanced partitions → log n depth → O(n log n) average; in-place, O(log n) stack.

Quick Sort — Complexity

Best	Average	Worst	Space
$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$

Average $O(n \log n)$ with balanced partitions. Worst case $O(n^2)$ when the pivot is always the smallest/largest (e.g. sorted input with a naive pivot). In place, so only $O(\log n)$ stack space.

Merge Sort vs Quick Sort

Merge Sort	Always O(n log n) · stable · needs O(n) extra space · great for linked lists / external sort
Quick Sort	O(n log n) average, O(n²) worst · in place O(log n) · usually fastest in practice

Homework

Estimate the complexity

Work out the best-case and average-case time complexity of Merge Sort by hand — count the levels of splitting and the work done at each level. Show why it comes out to O(n log n).

H1

Homework Solution

The Master method is a direct way to obtain the solution for recurrences that can be transformed to the type T(n) = aT(n/b) + O(n^k), where a ≥ 1 and b > 1. There are three cases for analysis using the master theorem:

- If f(n) = O(n^k) where k < logb(a), then T(n) = O(n^logb(a)).
- If f(n) = O(n^k) where k = logb(a), then T(n) = O(n^k * logn).
- If f(n) = O(n^k) where k > logb(a), then T(n) = O(n^k).

Let's compare with the merge sort recurrence:

- T(n) = 2T(n/2) + cn
- T(n) = aT(n/b) + O(n^k)

Activate Wi
Go to Settings t

H2

Homework Solution

Here, a = 2, b = 2 (a > 1 and b > 1).

- $O(n^k) = cn = O(n^1) \Rightarrow k = 1$
- Similarly, logb(a) = log2(2) = 1

So k = logb(a). This means that the merge sort recurrence satisfies the 2nd case of the master theorem. So merge sort time complexity $T(n) = O(n^k * \log n) = O(n^1 * \log n) = O(n \log n)$.

Homework Solution – Another Way

For N elements, Merge Sort divides the array into two equal halves:

$$N \to \frac{N}{2} + \frac{N}{2}$$

Each half is sorted recursively:

$$T\left(\frac{N}{2}\right) + T\left(\frac{N}{2}\right) = 2T\left(\frac{N}{2}\right)$$

After both halves are sorted, they are merged.
Merging N elements takes linear time:

$$cN$$

Therefore,

$$\boxed{T(N) = 2T\left(\frac{N}{2}\right) + cN}$$

Now expand:

$$T\left(\frac{N}{2}\right) = 2T\left(\frac{N}{4}\right) + c\frac{N}{2}$$

Substitute into $T(N)$:

$$T(N) = 2\left[2T\left(\frac{N}{4}\right) + c\frac{N}{2}\right] + cN$$

$$= 4T\left(\frac{N}{4}\right) + 2\left(c\frac{N}{2}\right) + cN$$

$$= 4T\left(\frac{N}{4}\right) + cN + cN$$

$$\boxed{T(N) = 4T\left(\frac{N}{4}\right) + 2cN}$$

Expand again:

$$T\left(\frac{N}{4}\right) = 2T\left(\frac{N}{8}\right) + c\frac{N}{4}$$

Substitute:

$$T(N) = 4\left[2T\left(\frac{N}{8}\right) + c\frac{N}{4}\right] + 2cN$$

$$= 8T\left(\frac{N}{8}\right) + 4\left(c\frac{N}{4}\right) + 2cN$$

$$= 8T\left(\frac{N}{8}\right) + cN + 2cN$$

$$\boxed{T(N) = 8T\left(\frac{N}{8}\right) + 3cN}$$

Homework Solution – Another Way

Continuing this pattern:

$$\boxed{T(N) = 2^k T\left(\frac{N}{2^k}\right) + kcN}$$

The division stops when only one element remains:

$$\frac{N}{2^k} = 1$$

$$N = 2^k$$

$$\log_2 N = \log_2(2^k)$$

$$\boxed{k = \log_2 N}$$

Substitute $k = \log_2 N$:

$$T(N) = 2^{\log_2 N} T\left(\frac{N}{2^{\log_2 N}}\right) + cN \log_2 N$$

Since

$$2^{\log_2 N} = N$$

and

$$\frac{N}{2^{\log_2 N}} = 1$$

we get:

$$T(N) = NT(1) + cN \log_2 N$$

Since $T(1)$ is constant:

$$T(N) = O(N) + O(N \log N)$$

Therefore,

$$\boxed{\therefore\ T(N) = O(N \log N)}$$

SECTION 04

Sorting a Linked List

The same two sorts, but on nodes and pointers instead of an array.

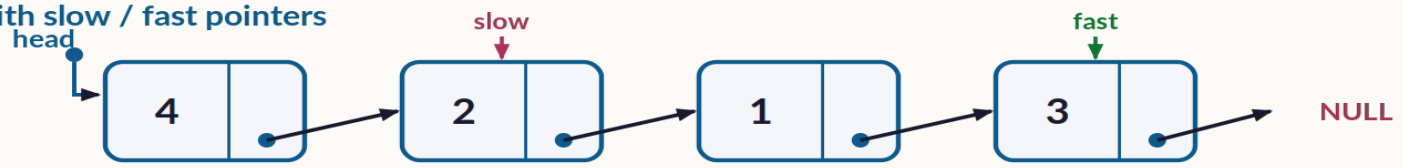
Why Lists Sort Differently

- A linked list has no indices — you can't jump to arr[mid] or arr[i] in O(1).
- So sorts that rely on random access (like array quick sort's indexing) need rethinking.
- But re-pointing next links is cheap, so merge sort fits linked lists beautifully.
- Both Merge Sort and Quick Sort can be adapted — by moving pointers, not copying values.

Merge Sort on a Linked List

Split with slow/fast pointers, sort each half, then merge by relinking nodes.

1. Split with slow / fast pointers



2. Two halves → sort each



3. Merge by relinking nodes



No array indices needed: find the middle with slow/fast pointers, split, sort each half, then merge by re-pointing next links.

Merging changes pointers only — no copying — so it needs no extra array. O(n log n) time.

Merge Sort on a List — Algorithm

Steps

1. If head is NULL or single node → sorted
2. Find the middle with slow / fast pointers
3. Split the list into two halves
4. Recursively merge-sort each half
5. Merge the two sorted halves by relinking

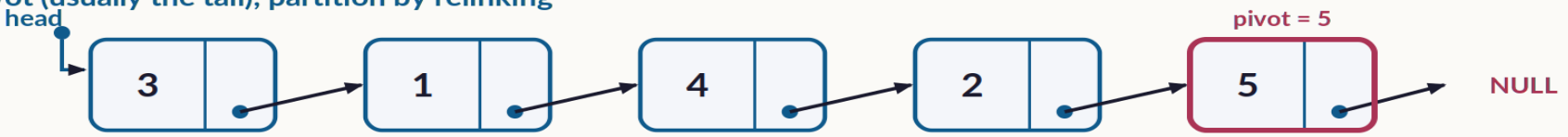
```
// pseudocode

mergeSortList(head):
    if head == NULL or head.next == NULL:
        return head
    mid = findMiddle(head)      // slow/fast
    right = mid.next; mid.next = NULL
    L = mergeSortList(head)
    R = mergeSortList(right)
    return mergeLists(L, R)      // relink
nodes
```

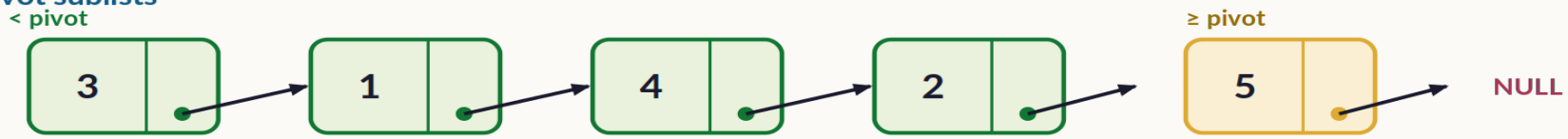
Quick Sort on a Linked List

Pick a pivot (usually head/tail), partition into two sublists, recurse, then join.

1. Pick pivot (usually the tail); partition by relinking



2. Build < pivot and ≥ pivot sublists



3. Recurse each side, then join: less → pivot → greater



Partition by moving nodes into two sublists, sort each, then concatenate. No random access, so the pivot is the head or tail.

In place (pointer moves only), but a bad pivot still gives $O(n^2)$; partitioning a list is fiddlier than an array.

Quick Sort on a List — Algorithm

Steps

1. If head is NULL or single node → sorted
2. Choose a pivot (commonly the head or tail)
3. Partition nodes into < pivot and ≥ pivot lists
4. Recursively quick-sort both sublists
5. Concatenate: less → pivot → greater

```
// pseudocode

quickSortList(head):
    if head == NULL or head.next == NULL:
        return head
    pivot = head
    (less, greater) = partition(head.next,
pivot)
    less    = quickSortList(less)
    greater = quickSortList(greater)
    return concat(less, pivot, greater)
```

Merge vs Quick — on a Linked List

Merge Sort

Preferred for lists: no random access needed, stable, guaranteed O(n log n), and merging is just pointer relinking (no extra array).

Quick Sort

Works in place, but no O(1) indexing makes partitioning fiddlier, the pivot is limited to head/tail, and it still degrades to $O(n^2)$ on bad pivots.

Bottom line: for linked lists, Merge Sort is the natural choice — it's what most standard library list sorts use.

SECTION 05

Practice It Yourself

These sorts power real interview problems — trace the recursion by hand.

Problems to Try

Draw the divide-and-conquer tree on paper before you code the recursion.

LeetCode 215	Kth Largest Element	Medium	Find the kth largest — QuickSelect (partition).
LeetCode 912	Sort an Array	Medium	Implement merge sort or quick sort from scratch.
LeetCode 148	Sort List	Medium	Merge sort on a linked list.
LeetCode 23	Merge k Sorted Lists	Hard	Divide-and-conquer merging.
LeetCode 493	Reverse Pairs	Hard	Count pairs during a merge sort.

LeetCode 215 — The Approach

Problem (in short): find the kth largest element in an unsorted array — ideally without fully sorting it.

- The kth largest sits at index n − k once the array is sorted.
- QuickSelect uses quick sort's partition, but recurses into only ONE side.
- After a partition, if the pivot lands at the target index, we're done.
- Else recurse only into the side that contains the target — average O(n).

YOUR TURN

On paper, partition [3,2,1,5,6,4] for k=2 (target index 4). Which side does QuickSelect recurse into?

LeetCode 215 — The Code

```
int quickSelect(vector<int>& a, int l, int r, int target) {
    int pivot = a[r], i = l;
    for (int j = l; j < r; j++)
        if (a[j] < pivot) swap(a[i++], a[j]);
    swap(a[i], a[r]);               // pivot in place at i
    if (i == target) return a[i];
    if (i < target)
        return quickSelect(a, i+1, r, target);
    return quickSelect(a, l, i-1, target);
}
int findKthLargest(vector<int>& a, int k) {
    return quickSelect(a, 0, a.size()-1, a.size()-k);
}
```

Why It Works & How Fast

One-sided recursion	Only the side holding the target is explored, unlike full quick sort.
Target index	kth largest = index n−k in sorted order; we partition until the pivot lands there.
Average time	O(n) — n + n/2 + n/4 + … ≈ 2n as the search halves each step.
Worst case	$O(n^2)$ with unlucky pivots; randomising the pivot makes it rare.

Where to Get Help

Watch the idea only after you've tried it — then close the video and code it yourself.

NeetCode	Kth Largest via QuickSelect. Search: "NeetCode Kth Largest Element"
Abdul Bari	Merge & quick sort with dry-runs. Search: "Abdul Bari Merge Sort" / "Quick Sort"
William Fiset	Divide-and-conquer sorting playlist. Search: "William Fiset Merge Sort"

YOUR TURN

Course rule: trace the recursion tree on paper for 20 minutes before watching anything — and never paste it into an AI.

LECTURE 7 — RECAP

What we covered

- Divide & conquer: split, solve, combine
- Merge sort: split to singles, merge sorted — O(n log n)
- Quick sort: partition around a pivot, recurse
- Quick sort O(n log n) avg, $O(n^2)$ worst, in place
- On linked lists: merge sort is the natural fit
- Pivot choice decides quick sort's speed

Next: Trees